SIT225 Data Capture Technologies
Assessment Pass Task 4
Protocol Execution & Product Demonstration

Estimated time: 10-15 hours Last updated: 14 June 2026

Due: End of Week 9, no later than week 10
Submit early, by week 9, to receive feedback

I.Instructions and Conditions

This task is due in **Week 9 (Sunday)**, and you are expected to start early. If your first submission is incomplete or incorrect, you may resubmit after receiving feedback. There will be **no extensions**, and late submissions will not be accepted. If your submission is not correct and complete by the deadline, it will be marked as a **FAIL**. This task is a **hurdle requirement**, failure to submit a correct version on time will result in failing the unit.

All submissions will be checked for **plagiarism** and **GenAI** use. *Any GenAI usage should be acknowledged (see template below) and you must include, in appendix, the prompt, raw response, and your changelog how you have adopted the GenAI response in your work.* You must complete the work **independently**, and you must not share or show your work to anyone else, unless it is explicitly instructed in your task to do so. If you have questions or encounter problems, attend the on-campus or online classes, or use the **Discussion Board** on the unit site.

Per Deakin policy, include an Acknowledgements section:

"This assessment work was developed with assistance from the following approved Deakin genAI tools:

- Deakin GEM (accessed June 2026): Used to <*generate structured critique on protocol design and confounding variables (3 iterations)*>.
- Microsoft Copilot for web (accessed June 2026): Used to <*summarise ASHRAE comfort standards and identify relevant literature.*>

All genAI-generated suggestions were critically evaluated and integrated only where supported by evidence. The final design, decisions, and justifications are my own."

II.Task Overview

This is the *capstone execution task* where you implement the refined use case you designed in Assessment 3. You will:

1. *Execute your data capture protocol* (live or simulated) based on Assessment 3 design updated by peer review process
2. *Process and analyse* collected data using statistical methods (predictive modelling optional)
3. *Demonstrate* your working system via a 3-minute video with face-on-camera narration and screen recording

This task assesses:

- ULO1: Acquire, process, visualise, analyse and interpret data [Weeks 1–8]
- ULO3: Design and implement data capture protocols [Weeks 5–9]
- ULO2 (implicit): Professional communication of findings [Week 9]

III.Task Components

Part 1: Protocol Execution & Data Acquisition (40% effort)

Implement your Assessment 3 design and collect real data.

1.1 Hardware & Software Setup

Based on your Assessment 3 protocol, set up your system:

Sensors & Platform:

- *Primary sensors:* DHT22, PIR, soil moisture, accelerometer, or equivalent [as per Assessment 3]
- *Communication:* Serial (Arduino → laptop via USB) or WiFi (Arduino IoT Cloud / Firebase)
- *Storage:*
    - Option A: Local CSV (simple, beginner-friendly)
    - Option B: Firebase Realtime Database [Week 5 course activity]
    - Option C: Arduino IoT Cloud [Week 3 course activity]

Required code components:

Component	*Technology*	*Reference*
Sensor reading	Arduino .ino sketch	Week 1–2 activities
Data ingestion	Python script (serial read or cloud download)	Week 5 Firebase activity
Buffering & timestamp	Pandas DataFrame	Week 2–5 activities
Storage	CSV file or cloud database	Week 3, 5 activities

Example code structure (Python):

```
import serial
import pandas as pd
from datetime import datetime

# Read from Arduino serial port
ser = serial.Serial('/dev/ttyACM0', 9600, timeout=1)
data = []

for i in range(duration_seconds):
    line = ser.readline().decode()
    timestamp = datetime.now().isoformat()
    reading = float(line.strip())
    data.append({'timestamp': timestamp, 'sensor_value': reading})

df = pd.DataFrame(data)
df.to_csv('my_data.csv', index=False)
```

1.2 Data Collection Execution

Execute your protocol *as designed in Assessment 3:*

- *Duration:* Minimum as specified in Assessment 3 (e.g., 3 months for seasonal comparison, 20–30 minutes for pilot)
    - *For this task:* Minimum *30 minutes of continuous collection* (or simulated equivalent if hardware unavailable)
- *Sensors:* All sensors specified in Assessment 2
- *Sampling frequency:* As per your protocol design
- *Quality assurance:* Apply the validation checks you planned (outlier removal, missing data handling)

Documentation required:

- Screenshot/photo of hardware setup
- Log of collection start/end time
- Any deviations from planned protocol (with explanation)
- Data quality report (% missing, outliers removed, final row count)

1.3 Data Processing Pipeline

Process raw data following Assessment 3 design:

Steps (using Pandas + NumPy):

1. *Load data* into Pandas DataFrame
2. *Validate:* Check for missing values, duplicates, data type consistency
3. *Clean:* Remove or interpolate missing data; flag/remove sensor errors (outliers beyond physical range)

4. *Transform:* Add derived columns (e.g., moving average, time-of-day, day-of-week) if relevant
5. *Aggregate:* If needed, downsample (e.g., 1-minute rolling averages from 10-second samples)
6. *Export:* Save processed CSV for analysis

Example Pandas workflow:

```
import pandas as pd
import numpy as np

df = pd.read_csv('raw_data.csv')

# Validate
print(f"Missing values: {df.isnull().sum()}")
print(f"Data shape: {df.shape}")

# Remove outliers (e.g., temperature <0 or >50)
df = df[(df['temperature'] > 0) & (df['temperature'] < 50)]

# Add moving average
df['temp_smooth'] = df['temperature'].rolling(window=5).mean()

# Save
df.to_csv('processed_data.csv', index=False)
```

Part 2: Statistical Analysis & Visualisation (35% effort)

Analyse your data to test the hypotheses from Assessment 3.

2.1 Descriptive Statistics

Compute and report:

Statistic	*Purpose*	*Example output*
Mean, median, std dev	Summarise distribution	Temp: μ=22.4°C, σ=1.8°C
Min, max, range	Identify extremes	Humidity range: 30–75%
Percentiles (25th, 75th, 95th)	Tail behaviour	95th percentile temp: 25.1°C
Correlation	Relationships between variables	temp vs. humidity: r=0.62

Output format:

```
print(df.describe())
print(f"Correlation matrix:\n{df.corr()}")
```

2.2 Visualisations (Minimum 3 plots)

Create *at least 3 distinct plots* using Plotly or Matplotlib:

Required plots:

Plot #	*Type*	*Purpose*	*Example*
1	*Time series line plot*	Show temporal pattern; identify trends	Temperature vs. time (line plot, colour by occupancy)
2	*Distribution/ histogram*	Characterise variable distribution	Histogram of humidity; bell curve overlay
3	*Relationship plot*	Test hypothesis about variable interaction	Scatter plot: temperature vs. comfort rating (colour by season)
Optional 4	*Heatmap / box plot*	Compare across categories	Box plot: temperature by time-of-day

Plotly example (interactive dashboard):

```
import plotly.express as px

# Time series
fig1 = px.line(df, x='timestamp', y='temperature', title='Temperature Over Time')

# Distribution
fig2 = px.histogram(df, x='humidity', nbins=30, title='Humidity Distribution')

# Scatter with regression
fig3 = px.scatter(df, x='temperature', y='comfort_rating', trendline='ols', title='Temperature vs.
User Comfort')
```

2.3 Statistical Testing & Hypothesis Validation

For each hypothesis from Assessment 2, perform relevant statistical test:

Example analysis (H1: "Temperature–comfort differs by season"):

```
from scipy import stats

# Separate by season
winter = df[df['season'] == 'winter']
summer = df[df['season'] == 'summer']

# Compare comfort ratings
t_stat, p_value = stats.ttest_ind(winter['comfort_rating'],
summer['comfort_rating'])
print(f"T-test: t={t_stat:.3f}, p={p_value:.4f}")

# Linear regression: comfort ~ temperature + season
from sklearn.linear_model import LinearRegression
X = df[['temperature', 'season_encoded']] # encode season as 0/1
y = df['comfort_rating']
model = LinearRegression()
model.fit(X, y)
print(f"R² = {model.score(X, y):.3f}")
```

```
print(f"Coefficients: temp={model.coef_[0]:.3f}, season={model.coef_[1]:.3f}")
```

Report findings:

- Did data support or refute each hypothesis?
- What is the effect size? (e.g., "A 1°C increase → 0.15-point comfort increase")
- Any unexpected patterns?

2.4 Data Interpretation Report (500–700 words)

Write a concise analysis:

1. *Summary of findings:* Key statistics and patterns observed
2. *Hypothesis validation:* Which hypotheses were supported? Confidence level (p-values, R^2 values)
3. *Limitations:* Data gaps, sensor accuracy, confounding variables, collection duration
4. *Implications:* What does this tell you about the original problem?
5. *Recommendations:* Refinements for future collection or deployment

Example (3 hypotheses):

> *H1 (temperature–comfort seasonality):* Supported. Winter comfort range (20–22°C) is narrower than summer (22–26°C). Effect significant: p=0.002. Suggests HVAC should use seasonal setpoint profiles.
>
> *H2 (occupancy prediction reduces waste):* Partially supported. Unoccupied rooms averaged 2.1°C drift from setpoint; with 15-min predictive trigger, estimated drift reduces to 0.8°C. However, only 3 weeks data; longer study needed.
>
> *H3 (humidity–discomfort):* Exploratory. Low correlation (r=0.18 between humidity and comfort ratings). Suggests humidity secondary to temperature. Limitation: only 10 comfort ratings (low n).

Part 3: Predictive Modelling (Optional, for higher confidence)

***Not required for pass**, but encouraged to strengthen findings.*

3.1 Recommended Approaches (Beginner-Friendly):

Method	*Use Case*	*Effort*	*Python Library*
Linear regression	Predict comfort from temp+humidity	Low	scikit-learn
Moving average	Forecast next 1–2 hours of temperature	Low	Pandas .rolling()
Classification (logistic)	Predict occupancy (on/off) from temp change	Medium	scikit-learn
Time series (ARIMA)	Model seasonal temperature patterns	Medium	statsmodels

Method	*Use Case*	*Effort*	*Python Library*
Simple neural net (optional)	Non-linear comfort prediction	Medium–High	TensorFlow / PyTorch

3.2 Simple Linear Regression Example:

```
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score, mean_squared_error

# Predict comfort from temperature
X = df[['temperature']].values
y = df['comfort_rating'].values

model = LinearRegression()
model.fit(X, y)

y_pred = model.predict(X)
r2 = r2_score(y, y_pred)
rmse = np.sqrt(mean_squared_error(y, y_pred))

print(f"R² = {r2:.3f}, RMSE = {rmse:.2f}")
print(f"Equation: comfort = {model.intercept_:.2f} + {model.coef_[0]:.3f} × temperature")
```

Part 4: 3-Minute Video Demonstration (25% effort)

Create a *professional 3-minute video* demonstrating your working system.

4.1 Video Requirements

- *Duration:* Exactly 3 minutes (±10 seconds)
- *Format:* MP4, 1080p minimum, clear audio
- *Recording method:*
    - Screen recording (OBS Studio, Zoom, QuickTime) + webcam/phone camera
    - Or: face-on-camera + laptop screen visible in shot
- *Audio:* Clear narration; no background noise
- *Submission:* MP4 file uploaded to learning management system (LMS)

4.2 Video Content Structure

Time	*Section*	*Content*
0:00–0:20	Introduction	Your name, unit, use case (1 sentence); show yourself on camera
0:20–0:50	Hardware setup	Screen: photo/diagram of sensors + Arduino; narrate: "I used DHT22 + PIR sensor connected via USB to my laptop…"
0:50–1:20	Data pipeline execution	Screen: Show running Python script capturing data in real-time; narrate: "Python script reads serial port every 5 seconds, buffers

Time	*Section*	*Content*
		data, and writes to CSV…"
1:20–2:00	Data analysis & visualisations	Screen: Show 3 plots (time series, distribution, scatter); narrate: "Plot 1 shows temperature rose 3°C over 30 minutes during occupancy. Plot 2 shows humidity concentrated around 50%. Plot 3 reveals positive correlation between temperature and comfort (r=0.62)."
2:00–2:40	Key findings & hypothesis testing	Screen: Show statistical output (R^2, p-values, correlation matrix); narrate: "H1: temperature–comfort differs by season (p=0.002, supported). H2: occupancy predicts energy waste (partial support, 3-week sample)."
2:40–3:00	Conclusion & next steps	Your face + screen: "This system demonstrates real-time sensor capture and analysis. Next iteration will use 3-month dataset and deploy cloud dashboard for live monitoring."

4.3 Production Tips

- *Narration:* Speak clearly; pace ~140 words/min (aim for 420 words total)
- *Screen layout:* Ensure font is readable (≥14pt)
- *Face visibility:* Good lighting; face fills ~¼ of frame if showing both face and screen
- *Editing:* Basic cuts/transitions OK; avoid excessive effects
- *Authenticity:* Show *your* work, face, and honest findings (don't fake data or over-dramatise)

4.4 Video Submission Checklist

- Video URL (panopto, or other online platform), 3:00 duration (±10 sec)
- Your face visible at intro & conclusion
- Screen recording shows: hardware setup, live script, 3 plots, statistical output
- Clear narration; no background noise
- All Assessment 2 hypotheses mentioned

IV.*Submission Checklist*

Component	*Format*	*Notes*
Week 8 activity	--	Perform each week's activity and export PDF to finally combine together with other items mentioned below. Clearly set heading of each week and activity number.
Makerspace Soldering induction	Completion certificate	You have to do Maker-space Soldering induction (online + physical) and append the completion certificate. Online students can submit only the online Soldering induction completion proof and submit a 2-4 page research report describing the soldering process and safety requirements..
Part 1: Execution & setup	PDF report + code + CSV	Hardware photos, Arduino .ino, Python script, raw + processed data

Component	*Format*	*Notes*
Part 2: Analysis report	PDF (500–700 words)	Descriptive stats table, 3+ visualisations, hypothesis testing results, interpretation
Part 3: Predictive model (optional)	Python script + output	If attempted: show R^2, RMSE, model coefficients, brief interpretation
Part 4: Video demonstration	Video online URL (3:00)	Face on camera, screen recording, narration, all hypotheses covered
Consolidate each part of the task into a single PDF, append previous weeks' (week 8) activity sheets at the bottom of the PDF. Append Maker-space Soldering induction completion certificate.		

Total submission package:

- 1 × PDF (Parts 1 & 2 combined)
- 2 × Python scripts (Arduino sketch, data analysis)
- 2 × CSV files (raw, processed)
- 1 × online recorded video
- Supporting files (hardware photos, optional model outputs)

V.*Grading Criteria (Pass/Fail)*

Criterion	*Pass*	*Notes*
Protocol execution	Data collected for ≥30 min; all Assessment 2 sensors/sampling used; setup documented	Hardware or realistic simulation acceptable
Data processing	Raw → processed pipeline evident; validation applied (outlier removal, missing data); Pandas code present	Code must be reproducible
Statistical analysis	≥3 plots generated; descriptive stats computed; ≥1 hypothesis tested with p-value or R^2 reported	Stats must relate to Assessment 2 hypotheses
Interpretation	Findings clearly connect to original problem; limitations acknowledged; recommendations proposed	Avoid vague conclusions
Video quality	3:00 duration; face visible; screen clear; narration audible; all hypotheses mentioned	Production quality (editing, lighting) secondary to content clarity
Professionalism	Code commented; plots labelled; report well-structured; video authentic (no fabricated data)	Show genuine work; honest about limitations

Pass awarded when: All criteria met to satisfactory standard. Predictive modelling not required but encouraged.

VI.*Common Pitfalls to Avoid*

- Collecting <10 minutes of data (insufficient for pattern detection)
- Skipping data validation (outliers or missing values unaddressed)

- Plots without labels, units, or titles
- Hypothesis testing without reporting p-values or effect sizes
- Video >3:10 or <2:50 (fails submission criteria)
- Face not visible in video (key requirement)
- Narration unclear or rushed (speak ~140 words/min)
- Fabricating data or findings (assessment integrity)